

Programming for Biologists: An Introduction to Python and R

DATA405 Genomic Data Science • Lecture 10 • Sep 28, 2026

ATGCCGGAGGCCCGCCTCTGCTGTTGGCAGCTGTGTTGCTGGGCCTGGTGCTGCTGGTGGTG

Jinrui Xu, Department of Biology, Howard University

Every example today runs on the course website in your browser — nothing to install.

Why a biologist learns to program

- **The genome is text.** 3.2 billion letters; a sequencing run is millions of 100-letter strings. No spreadsheet opens that.
- **Your question is new.** A tool exists for the common step (mapping, counting). It does not exist for the question in your proposal.
- **Reproducibility.** A script is a lab protocol that runs itself, identically, on the next sample.
- **Reading.** Methods sections, GitHub, Bioconductor vignettes — you must read code even if you rarely write it.

Goal today: after two hours you can read a short Python or R script and write a few lines yourself.

What a program is

- A program is a recipe: a list of instructions, executed top to bottom, one at a time.
- An interpreter (Python or R) reads each line and does exactly what it says — nothing more, nothing less.
- The computer never guesses. A missing quote, a wrong letter, an off-by-one number: it either stops with an error, or (worse) does the wrong thing silently.
- **Two languages, one habit of mind:** state what you want precisely, then check the result against something you already know.

Today

	Minutes	What you will be able to do
Part A · Python from zero	70	values, strings (DNA), lists, dictionaries (the genetic code), if, loops, functions, files, pandas tables, a plot
Part B · R, by translation	40	vectors, strings, data frames, base-R plots (CADSA recipe), a t-test in one line
Part C · Both	10	when to use which; the five bugs; ChatGPT rules; the lab

Predict before I run. Every YOUR TURN is answered out loud before the reveal.

Part A

Python from zero

Guido van Rossum, 1991. Designed to read like English. The language of pipelines, machine learning and most of GitHub.

Your first line

A cell holds instructions. Run it; Python prints what you asked. Text goes in quotes; numbers do not.

Python

```
print("Hello, genome")
```

```
# → Hello, genome
```

```
print(3 + 4)
```

```
# → 7
```

```
print("COMT", "has", 271, "amino acids")
```

```
# → COMT has 271 amino acids
```

print(...) shows a value. Anything after **#** is a comment for humans; Python ignores it.

Variables: give a value a name

Python

```
gene = "COMT"           # text (a string)
n_aa = 271               # a whole number (an integer)
gc = 0.618               # a decimal (a float)

print(gene, n_aa, gc)
# → COMT 271 0.618

n_nt = n_aa * 3 + 3      # arithmetic with a variable
print(n_nt)
# → 816

type(gene), type(n_aa), type(gc)
# → str, int, float
```

= means 'store this value under this name'. A variable is a labelled box; you can reuse and overwrite it.

Predict

YOUR TURN What does this print?

- `x = 5`
- `x = x + 1`
- `print(x * 2)`

ANSWER 12. The second line replaces the old value (5) with 6; then $6 \times 2 = 12$. Read = as 'becomes', not 'equals'.

Strings are text — and DNA is text

Python

```
dna = "ATGCCGGAGGCC"
```

```
len(dna)           # → 12           how many letters
dna.count("G")      # → 5           how many G
dna.upper()         # → 'ATGCCGGAGGCC'
dna.replace("T", "U") # → 'AUGCCGGAGGCC'  DNA → RNA
"ATG" in dna        # → True        is there a start codon?
dna + "TGA"         # → 'ATGCCGGAGGCCTGA' join text
```

Methods are actions attached to a value: `dna.count(...)` asks the string to count. Nothing here changes `dna` itself.

Indexing: the first letter is number 0

Python

```
dna = "ATGCCGGAGGCC"
```

```
#      0123456789...
```

```
dna[0]      # → 'A'    first letter
```

```
dna[1]      # → 'T'
```

```
dna[-1]     # → 'C'    last letter
```

```
dna[0:3]    # → 'ATG'  letters 0,1,2 (3 is NOT included)
```

```
dna[3:6]    # → 'CCG'  the second codon
```

```
dna[:3]     # → 'ATG'  from the start
```

```
dna[9:]     # → 'GCC'  to the end
```

Python counts from 0, and a slice [a:b] stops just before b. This one rule causes half the coordinate bugs in genomics.

0 or 1?

YOUR TURN Codon 158 of COMT starts at which Python index?

- A. $158 \times 3 = 474$
- B. $(158 - 1) \times 3 = 471$
- C. 158
- D. 157

ANSWER B. Codon 1 starts at index 0, codon 2 at 3, ... codon k at $(k - 1) \times 3$. So `cds[471:474]` is codon 158. GTF and VCF files count from 1 — subtract one when you cross over.

Lists: many values in order

Python

```
genes = ["COMT", "BCR", "CHEK2", "NF2"]

len(genes)          # → 4
genes[0]             # → 'COMT'
genes[-1]            # → 'NF2'
genes[1:3]           # → ['BCR', 'CHEK2']

genes.append("SOX10") # add at the end
sorted(genes)         # → ['BCR', 'CHEK2', 'COMT', 'NF2', 'SOX10']
"BCR" in genes        # → True

lengths = [816, 1260, 28204]
sum(lengths) / len(lengths) # → 10093.3  the mean
```

A list is an ordered collection in square brackets. Same indexing rules as strings: 0-based, slices stop before the end.

Dictionaries: look something up by name

Python

```
# the genetic code IS a dictionary: codon → amino acid
code = {"ATG": "M", "GTG": "V", "GAG": "E", "TAA": "*"}

code["ATG"]          # → 'M'
code["GTG"]          # → 'V'
code["TTT"]          # → KeyError: 'TTT'    (not in the table)

code["TTT"] = "F"    # add an entry
len(code)            # → 5
list(code.keys())    # → ['ATG', 'GTG', 'GAG', 'TAA', 'TTT']
```

Curly braces, key: value pairs. A dict answers 'what does X map to?' in one step — exactly what translation needs.

Predict

YOUR TURN sample = {"IGLC2": 147872, "COMT": 922, "BCR": 0} — what is sample["COMT"] + sample["BCR"]?

- A. 922
- B. 'COMTBCR'
- C. KeyError
- D. 148794

ANSWER A. Both keys exist; $922 + 0 = 922$. If you had typed sample["comt"] (lower case) you would get a KeyError — dictionaries are exact.

True/False and if: making decisions

Python

```
q = 35                # a Phred quality score

q >= 30               # → True    (a Boolean)

if q >= 30:
    print("good base")
elif q >= 20:
    print("acceptable")
else:
    print("discard")
# → good base

# combine conditions
q >= 20 and q < 30    # → False
not (q >= 30)         # → False
```

The colon starts a block; the indented lines belong to it. Indentation is not decoration in Python — it is the grammar.

Loops: do something for every item

Python

```
for base in "ATG":  
    print(base)  
# → A  
# → T  
# → G  
  
total = 0  
for x in [816, 1260, 28204]:  
    total = total + x  
print(total)          # → 30280  
  
for i in range(3):    # 0, 1, 2  
    print(i)
```

for item in collection: — the block runs once per item. An accumulator (total) starts at 0 and grows each round.

A loop that computes GC content

Combine a loop, an if, and an accumulator. This is the shape of most small analysis programs.

Python

```
dna = "ATGCCGGAGGCC"

gc_count = 0
for base in dna:
    if base == "G" or base == "C":
        gc_count = gc_count + 1

gc = gc_count / len(dna)
print(gc)                # → 0.75
```

Trace it by hand for 'ATGC': gc_count goes $0 \rightarrow 0 \rightarrow 0 \rightarrow 1 \rightarrow 2$; $2/4 = 0.5$. Tracing by hand is how you find bugs.

Predict

YOUR TURN What does the loop print for `dna = "AAAA"`?

- A. 0
- B. 0.0
- C. 4
- D. an error

ANSWER B. `gc_count` stays 0; `0 / 4` is the float 0.0. (A division always gives a float in Python 3.)

Functions: name a recipe so you can reuse it

Python

```
def gc_content(dna):  
    """Fraction of letters that are G or C."""  
    g = dna.count("G")  
    c = dna.count("C")  
    return (g + c) / len(dna)
```

```
gc_content("ATGCCGGAGGCC")    # → 0.75  
gc_content("AAAA")             # → 0.0  
gc_content("GGCC")             # → 1.0
```

def name(inputs): ... return output. Test a new function on cases whose answer you know (0.0 and 1.0) before trusting it on real data.

Files: reading a FASTA

A FASTA file: a header line starting with >, then the sequence over many lines.

Python

```
def read_fasta(path):
    seq = []                # collect lines here
    for line in open(path): # one line at a time
        if not line.startswith(">"):
            seq.append(line.strip()) # drop the newline
    return "".join(seq)     # glue into one string

cds = read_fasta("data/COMT_cds.fa")
len(cds)           # → 816
cds[:30]           # → 'ATGCCGGAGGCCCGCCTCTGCTGTTGGCA'
gc_content(cds)    # → 0.618
```

Everything above is things you already know: a loop, an if, a list, a string method, a function.

The genetic code as a dictionary (all 64 codons)

Python

```
bases = "TCAG"
aas = ("FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRR"
       "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG")
codon_table = {}
i = 0
for a in bases:
    for b in bases:
        for c in bases:
            codon_table[a + b + c] = aas[i]
            i = i + 1

codon_table["ATG"]      # → 'M'
codon_table["TAA"]      # → '*'    stop
```

Three nested loops walk TTT, TTC, TTA, ... in the standard order; the 64-letter string lists the amino acids in that order.

Translate COMT, and find rs4680

Python

```
def translate(dna):
    protein = ""
    for i in range(0, len(dna) - 2, 3):      # 0, 3, 6, ...
        protein = protein + codon_table[dna[i:i+3]]
    return protein

protein = translate(cds)
protein[:10]          # → 'MPEAPPLLLA'
len(protein) - 1      # → 271  (+ the stop)

i = (158 - 1) * 3      # codon 158 → index 471
cds[i:i+3]            # → 'GTG'  valine
mutant = cds[:i] + "A" + cds[i+1:]
translate(mutant)[157] # → 'M'      methionine: Val158Met
```

Twelve lines of Python reproduce a result from the literature: rs4680 turns valine 158 into methionine. You wrote every line.

Reading an error message

- **NameError: name 'dan' is not defined** — a typo in a variable name (dna).
- **IndexError: string index out of range** — you asked for letter 12 of a 12-letter string (the last one is 11).
- **KeyError: 'TTN'** — the codon has an N; it is not in the dictionary.
- **IndentationError** — a block is not indented consistently (mixed copy-paste).
- Read the last line first. Then the line number. Errors are the interpreter being honest — the silent wrong answer is the dangerous one.

Libraries: code other people wrote

- Python itself is small. Most power comes from packages you import: pandas (tables), matplotlib (plots), numpy (numbers), scipy (statistics), Biopython (sequences).
- **import pandas as pd** — load the package once, call it pd.
- Installed with `pip install pandas` (or conda). On the course website they are already there.
- The Bioconductor equivalent in R is where DESeq2 lives — Part B.

pandas: a table of all chr22 genes

Python

```
import pandas as pd
genes = pd.read_csv("data/chr22_genes.tsv", sep="\t")

genes.shape          # → (1747, 7)    rows, columns
genes.head(3)        # first three rows
genes["gene_type"].value_counts().head(3)
# → lncRNA 747, protein_coding 447, processed_pseudogene 142

pc = genes[genes["gene_type"] == "protein_coding"]    # filter rows
len(pc)                                               # → 447
pc["length_bp"].median()                             # → 23780.0
pc.sort_values("length_bp", ascending=False).head(3)
# → LARGE1 760660, TTC28 701827, SYN3 550562
```

A DataFrame is a table with named columns. `genes["col"]` picks a column; `genes[condition]` keeps the rows where the condition is True.

pandas: group, summarize, join

Python

```
genes.groupby("gene_type")["length_bp"].median().head(3)
# median gene length per type

expr = pd.read_csv("data/chr22_expression.tsv", sep="\t")
m = genes.merge(expr[["gene_id", "TPM"]], on="gene_id")

on = m[m["TPM"] > 1]
on["gene_type"].value_counts().head(3)
# which kinds of genes are expressed in this sample?

expr.sort_values("TPM", ascending=False).head(3)
# → IGLC2, IGLV2-23, ACTBP15
```

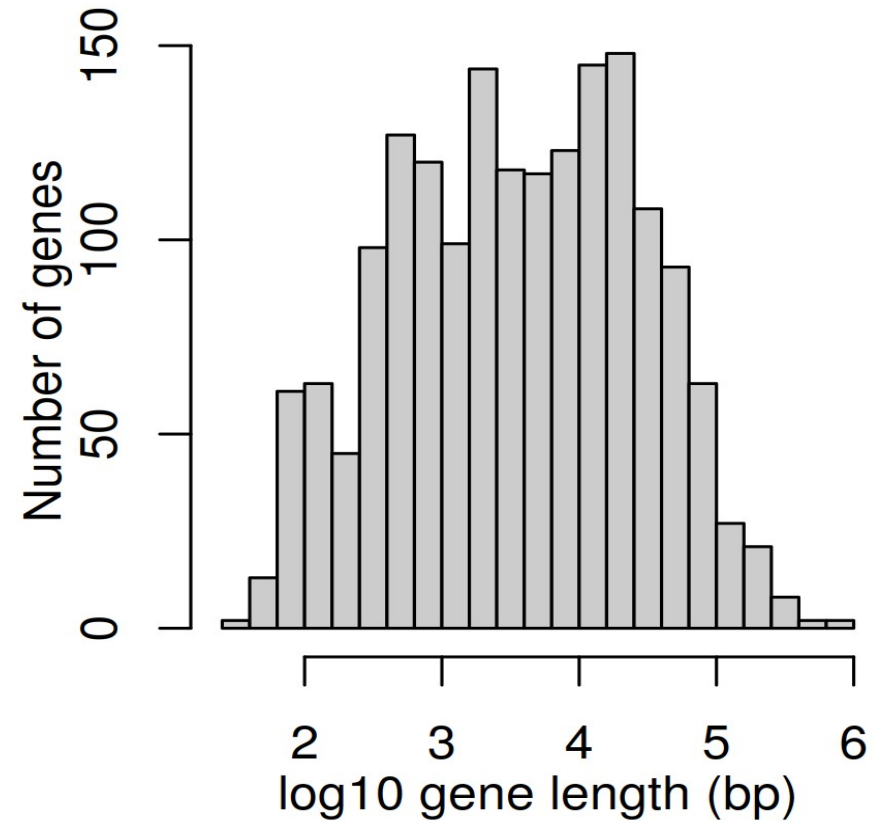
Five verbs — read, filter, sort, group/summarize, join — cover most analysis. The top genes are immunoglobulins: a B-cell sample (L9, from a table).

matplotlib: one plot

Python

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(3.2, 3.2)) # inches
ax.hist(np.log10(genes["length_bp"]), bins=30,
color="grey")
ax.set_xlabel("log10 gene length (bp)")
ax.set_ylabel("Number of genes")
plt.tight_layout()
plt.show()
```



Set the figure size in inches first — the same rule as in R (Part B).

Python in one slide

- **Values:** numbers, strings ("ATG"), True/False.
- **Collections:** list [a, b, c] (ordered, 0-based), dict {key: value} (look-up).
- **Control:** if / elif / else; for item in collection; blocks by indentation.
- **Functions:** def name(inputs): ... return output.
- **Files & libraries:** open(), import pandas as pd, matplotlib.
- **Habit:** test on a case you know; read the last line of an error.

You now know enough Python to read most bioinformatics scripts on GitHub.

Part B

R, by translation

Ross Ihaka & Robert Gentleman, 1993, from Bell Labs' S (1976). Built for statistics; home of Bioconductor and DESeq2.

Why a second language?

- R was designed by statisticians: a t-test, a linear model and a plot are each one line, built into the language.
- Bioconductor (2001): DESeq2, edgeR, limma, Seurat — the statistical methods of genomics are published here first.
- Most of what you just learned transfers directly. Only four things change: <- for =, counting from 1, vectors instead of loops, braces instead of indentation.
- You will use RStudio (or the course website). Same on Windows and Mac.

Learn R as a dialect of what you already know, not as a new language.

First lines in R

R

```
print("Hello, genome")  
# → [1] "Hello, genome"
```

```
3 + 4  
# → [1] 7
```

```
gene <- "COMT"           # <- is R's assignment (= also works)  
n_aa <- 271  
cat(gene, "has", n_aa, "amino acids\n")  
# → COMT has 271 amino acids
```

The [1] means 'this is element 1 of a vector' — in R, even a single number is a vector of length 1.

Vectors: R's unit of thought

R

```
lengths <- c(816, 1260, 28204)    # c() combines values
length(lengths)                  # → 3
lengths[1]                       # → 816      the FIRST element is 1
lengths[2:3]                     # → 1260 28204

lengths / 1000                   # → 0.816 1.260 28.204   whole vector at once
lengths > 1000                   # → FALSE TRUE TRUE
lengths[lengths > 1000]          # → 1260 28204   filter by condition
sum(lengths); mean(lengths); max(lengths)
```

In Python you loop over a list. In R you operate on the whole vector in one line. Most R loops you would write are unnecessary.

Translate

YOUR TURN Python: `[x / 1000 for x in lengths if x > 1000]` — the R version is...

- A. `for (x in lengths) if (x > 1000) x / 1000`
- B. `lengths[lengths > 1000] / 1000`
- C. `lengths / 1000 > 1000`

ANSWER B. Filter with a logical vector inside `[ ]`, then divide the whole result. One line, no loop.

Strings and DNA in R

R

```
dna <- "ATGCCGGAGGCC"
nchar(dna)                # → 12    (not length: that is 1!)
substr(dna, 1, 3)         # → "ATG"   letters 1 to 3, both included
substr(dna, 4, 6)         # → "CCG"
toupper(dna); gsub("T", "U", dna)
grepl("ATG", dna)         # → TRUE    is the pattern there?
paste0(dna, "TGA")        # join text

bases <- strsplit(dna, "")[[1]] # split into a vector of letters
bases                        # → "A" "T" "G" "C" ...
sum(bases == "G")           # → 5
mean(bases %in% c("G", "C")) # → 0.75   GC content, no loop
```

Turn the string into a vector of letters (strsplit), then every question becomes a vector question.

Named vectors: R's dictionary

R

```
code <- c(ATG = "M", GTG = "V", GAG = "E", TAA = "*")
code["GTG"]          # → "V"
names(code)          # → "ATG" "GTG" "GAG" "TAA"
code[c("ATG", "GTG")] # look up several at once → "M" "V"

# the full table, same idea as in Python
b <- c("T", "C", "A", "G")
aas <- strsplit(paste0("FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRR",
  "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"), "")[[1]]
codons <- character(0)
for (x in b) for (y in b) for (z in b)
  codons <- c(codons, paste0(x, y, z))
codon_table <- setNames(aas, codons)
codon_table["ATG"]  # → "M"
```

A vector with names does what a Python dict does — and it can look up a whole vector of codons in one call.

if, for, function — the same ideas with braces

R

```
q <- 35
if (q >= 30) {
  cat("good base\n")
} else if (q >= 20) {
  cat("acceptable\n")
} else {
  cat("discard\n")
}

for (base in c("A", "T", "G")) cat(base, "\n")

gc_content <- function(dna) {
  bases <- strsplit(dna, "")[[1]]
  mean(bases %in% c("G", "C"))      # last value is returned
}

gc_content("ATGCCGGAGGCC")      # → 0.75
```

Condition in (), block in { }. A function returns its last expression; return() is optional.

Read FASTA and translate COMT in R

R

```
read_fasta <- function(path) {  
  lines <- readLines(path)  
  paste(lines[!startsWith(lines, ">")], collapse = "")  
}  
cds <- read_fasta("data/COMT_cds.fa")  
nchar(cds) # → 816  
  
translate <- function(dna) {  
  n <- nchar(dna) %/% 3  
  starts <- seq(1, by = 3, length.out = n) # 1, 4, 7, ...  
  paste(codon_table[substring(dna, starts, starts + 2)], collapse = "")  
}  
protein <- translate(cds)  
substr(protein, 1, 10) # → "MPEAPPLLLA"  
i <- (158 - 1) * 3 + 1 # codon 158 → position 472  
substr(cds, i, i + 2) # → "GTG"  
mutant <- cds; substr(mutant, 1, 1) <- "A"  
substr(translate(mutant), 158, 158) # → "M"
```

Same result as Python. Note the vector move: compute all codon start positions, look them all up at once.

Data frames: the table, built in

R

```
genes <- read.delim("data/chr22_genes.tsv")
dim(genes)                # → 1747 7
head(genes, 3)
genes$gene_type[1:3]      # $ picks a column
table(genes$gene_type)    # counts per category

pc <- genes[genes$gene_type == "protein_coding", ] # rows, cols
nrow(pc)                  # → 447
median(pc$length_bp)      # → 23780
head(pc[order(-pc$length_bp), c("gene_name", "length_bp")], 3)
# → LARGE1 760660, TTC28 701827, SYN3 550562
```

df[rows, columns] — the comma matters. **df\$col** is a plain vector, so everything from the vector slide applies.

Group, summarize, join

R

```
tapply(genes$length_bp, genes$gene_type, median)
# median length per type (or: aggregate(length_bp ~ gene_type, genes, median))

expr <- read.delim("data/chr22_expression.tsv")
m <- merge(genes, expr[, c("gene_id", "TPM")], by = "gene_id")

on <- m[m$TPM > 1, ]
head(sort(table(on$gene_type), decreasing = TRUE), 3)

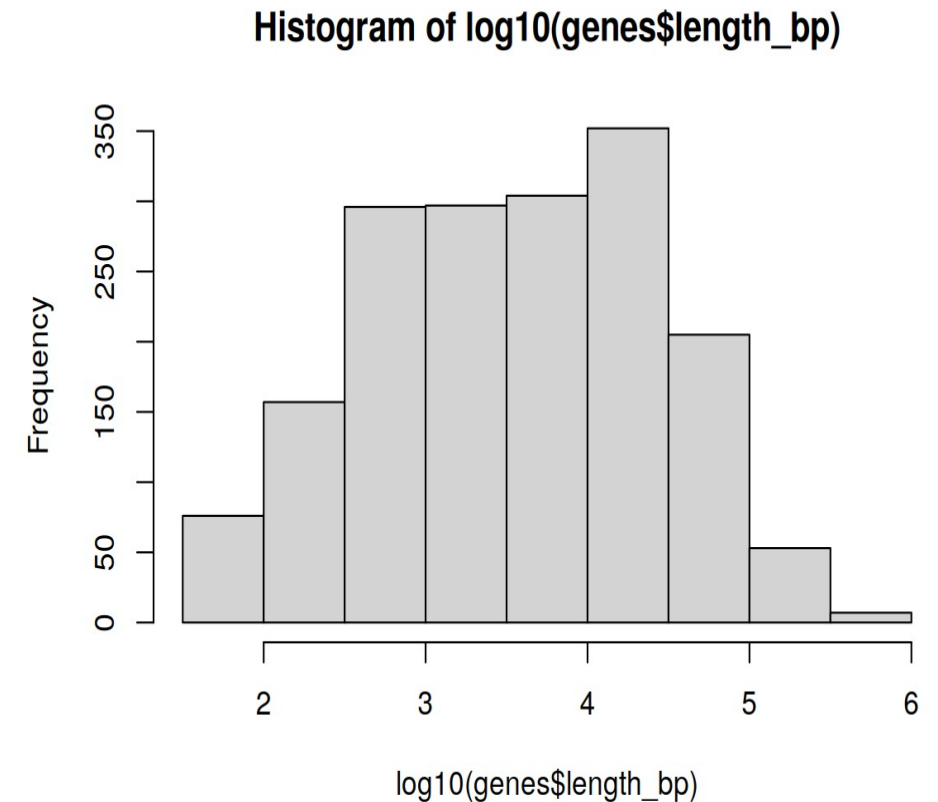
head(expr[order(-expr$TPM), c("gene_name", "TPM")], 3)
# → IGLC2 147873, IGLV2-23 93432, ACTBP15 92402
```

pandas groupby → tapply/aggregate; merge → merge; sort_values → order. The five verbs again.

Plotting is built in — but defaults are for the screen

R

```
hist(log10(genes$length_bp))
```

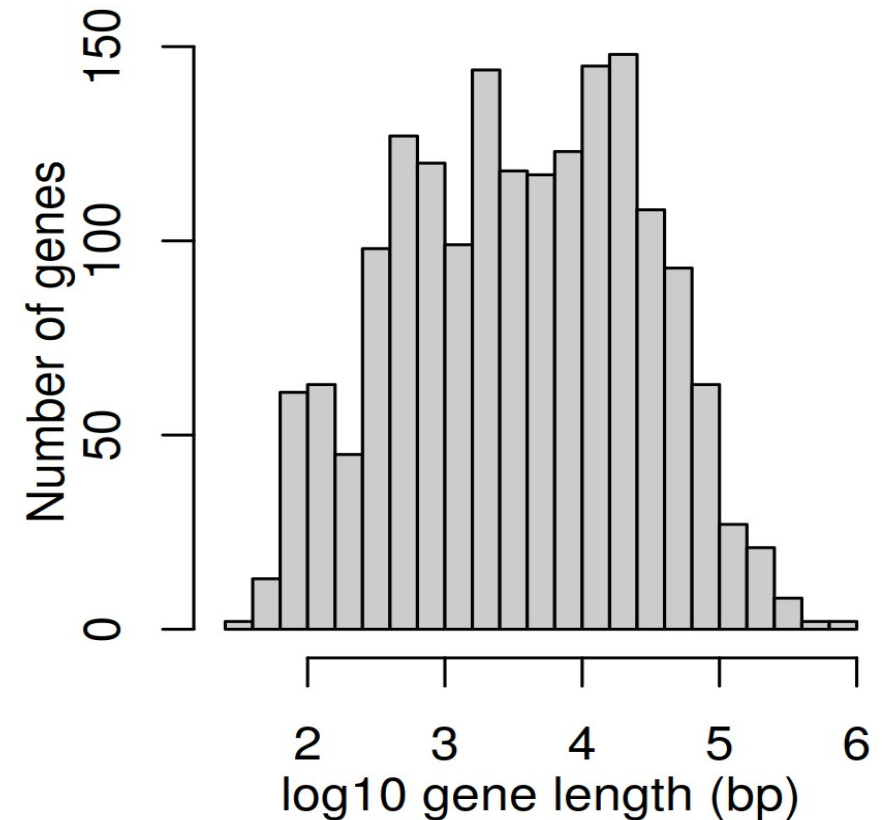


One line, and it works. Shrunk to a journal column, the labels vanish — the CADSA workshop lesson.

The CADSA recipe: think in inches first

R

```
png("fig1.png", width = 3.2, height = 3.2, units = "in",
    res = 300)
par(mar = c(4, 4, 1, 1))          # margins:
bottom, left, top, right
hist(log10(genes$length_bp), breaks = 30, col = "grey80",
     main = "", xlab = "", ylab = "", axes = FALSE) #
draw nothing automatic
axis(1, line = 0.1); axis(2, line = 0.1) # add axes
yourself
title(xlab = "log10 gene length (bp)", line = 2)
title(ylab = "Number of genes", line = 2)
dev.off()
```



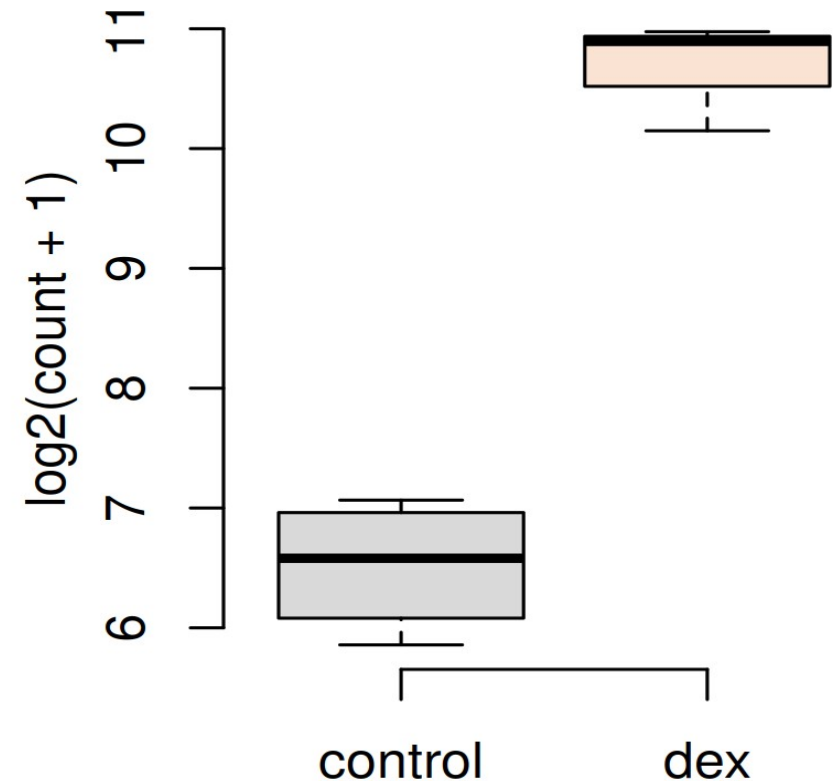
Canvas size → margins → suppress defaults → add axes and labels with cex you control. Same recipe for plot(), boxplot(), barplot().

Statistics is one line: a t-test on real counts

R

```
cts <- read.delim("data/airway_counts.tsv", row.names = 1)
meta <- read.delim("data/airway_metadata.tsv")
logc <- log2(cts + 1)
ctrl <- meta$id[meta$dex == "control"]
trt <- meta$id[meta$dex == "treated"]

g <- "ENSG00000152583" # SPARCL1
a <- unlist(logc[g, ctrl]); b <- unlist(logc[g, trt])
boxplot(list(control = a, dex = b))
t.test(b, a)
# → t = 12.5, df = 6, p-value = 3e-05
# → 95% CI of the difference: 3.3 to 4.9 (log2 units ≈ 16-fold)
```



Four donors, ± the steroid dexamethasone (Himes 2014). R prints the test, the CI and the means in one call. Tomorrow: why this test is still the wrong tool for 19,271 genes.

R in one slide

- **Vectors:** `c(...)`, 1-based, whole-vector arithmetic, filter with `v[condition]`.
- **Strings:** `nchar`, `substr`, `strsplit`, `paste`; named vector = dictionary.
- **Control:** `if ( ) { }`, `for ( ) { }`, `function( ) { }`.
- **Tables:** `read.delim`, `df$col`, `df[rows, cols]`, `order`, `table`, `tapply`, `merge`.
- **Plots & stats:** `hist`/`plot`/`boxplot` with the inches recipe; `t.test`, `lm`, `y ~ group` formulas.
- **Packages:** `install.packages()`, `BiocManager::install("DESeq2")`, `library()`.

You can now read a DESeq2 vignette. Tomorrow you will understand what it computes.

Part C

Both languages

When to use which, the bugs that bite everyone, and how to use ChatGPT without being used by it.

The Rosetta stone (the full table is on the site)

Task	Python	R
assign	<code>x = 5</code>	<code>x <- 5</code>
first letter / element	<code>s[0]</code>	<code>substr(s, 1, 1) / v[1]</code>
length	<code>len(s)</code>	<code>nchar(s) / length(v)</code>
count a letter	<code>s.count("G")</code>	<code>sum(strsplit(s, "")[[1]] == "G")</code>
look-up table	<code>{"ATG": "M"}</code>	<code>c(ATG = "M")</code>
filter a table	<code>df[df["x"] > 1]</code>	<code>df[df\$x > 1,]</code>
group + summarize	<code>df.groupby("g")["x"].median()</code>	<code>tapply(df\$x, df\$g, median)</code>
t-test	<code>scipy.stats.ttest_ind(a, b)</code>	<code>t.test(a, b)</code>
install	<code>pip install biopython</code>	<code>BiocManager::install("DESeq2")</code>

Which language for which step?

The data are...	Think...	Reach for...
Sequences, reads, files, a tool to glue	text	Python (or the shell)
One row per gene, columns of numbers	table	either; R if statistics come next
Counts with replicates and a hypothesis	table + uncertainty	R: DESeq2, edgeR, limma
Images, deep learning, single cell at scale	arrays	Python: PyTorch, scanpy
A figure for the paper	inches	base R (or matplotlib) with the CADSA recipe

Most real projects: a Python/shell pipeline produces a count table; an R notebook tests and plots it.

The five bugs that bit real papers

Bug	What happens	Defence
0- vs 1-based	Off by one base: wrong codon, wrong variant	Know which base is 0 for every file format
Strand	A minus-strand k-mer never matches	Reverse-complement before searching
Reference build	GRCh37 positions in a GRCh38 file (rs4680: 19,951,271 vs 19,963,748)	Write the build in every file name
Excel gene names	SEPT7 → 7-Sep, MARCH1 → 1-Mar (≈20 % of supplementary tables; genes renamed in 2020)	Never open a gene list in Excel
Silent type coercion	A numeric column read as text; p-values sorted as strings	Check dtypes / str(df) after every read

None of these crash. All of them produce a confident, wrong table.

ChatGPT writes code. You verify it.

- TRACE prompting, as in your proposal: Task, Role, Audience, Context, Example.
- **Then test on a case whose answer you already know:** COMT codon 158 must be GTG; the protein must start with M and have 271 aa; the reverse complement of TGA must be TCA.
- Ask it to explain every line. If you cannot explain a line, do not run it on real data.
- Copilot-style autocomplete is off in this lab: the exercises are exactly the lines it would type for you.

Lab 1 ends with a ChatGPT function that fails two of three tests. Your job will be to find out why.

Lab 1 on Wednesday (1.5 h, browser only)

- Ex 1 Hello, two languages • Ex 2 COMT: GC, reverse complement, translate, rs4680 • Ex 3 FASTQ quality
- Ex 4 Tables: genes, expression, mismatches (which look like real variants?)
- Ex 5 The t-test and the 19,271-tests trap • Ex 6 Catch ChatGPT's bug
- Every exercise in both languages, with hints and solutions on the site; the handout has the same exercises with blanks.
- Lab report: answers to Q1–Q7, the fixed function in both languages, one 3.2-inch figure.

Questions?

jinrui.xu@howard.edu

Course site: Lecture 10 notes, Rosetta stone, Lab 1, data, local setup for Windows and Mac.